Introduction Rapide aux Commandes Linux

1 Introduction

Le contenu de ce TP est un condensé du cours « Introduction au Shell » du module « Environnement Informatique » en Peip1, initialement écrit par Stéphane Lavirotte.

Ce TP est destiné à vous familiariser avec les fichiers sous l'environnement UNIX. Fortement axé sur la pratique, il aborde la création, la manipulation et la suppression des répertoires et fichiers. Même s'il est possible aujourd'hui d'utiliser quotidiennement Linux par une interface graphique, l'automatisation des taches par la programmation et la manipulation du système à distance entrent souvent en contradiction avec l'utilisation d'une interface graphique.

Étant donné que dans ce cours d'Introduction à la Programmation et à l'Algorithmique (IPA) nous allons devoir analyser, compiler et exécuter des programmes avec le langage de programmation C, il est essentiel de connaître au moins les bases de l'interface de commandes Linux.

2 Commandes & Cie

2.1 Interpréteur de commandes

Qu'est-ce que c'est un interpréteur de commandes ? Ce sont des outils pour exécuter des commandes tapées au clavier par un utilisateur dans un terminal. Les interpréteurs de commandes sont aussi appelés « shells » (coquilles).

Les commandes sont tapées dans un terminal en mode texte, constitué par une fenêtre dans un environnement graphique, ou par une console sur un écran en texte seul (sans environnement graphique).

En Ubuntu, tapez sur la touche Windows (la touche avec le logo de Windows en fait) pour ouvrir le menu principale et tapez « Terminal ». Le logo de l'application terminal devrait apparaître et vous n'avez qu'à cliquer dessus pour l'ouvrir et lancer vos commandes.

2.2 Commande

Une commande Unix est un mot ou une phrase à la syntaxe bien particulière entré dans un éditeur ou interpréteur de commandes et donnant l'ordre d'actions à exécuter par l'ordinateur.

Une commande UNIX se décompose en trois parties :

1. la commande elle-même,
2. des options, qui comme son nom l'indique, sont optionnelles (zéro, une ou plusieurs options),
3. des arguments : zéro, un nombre fixe ou variable d'arguments.

Par exemple, si vous voulez connaître une date, tapez dans un terminal la commande date et le système vous retournera un texte contenant la date du jour et l'heure. Avec l'option « -d », on demande à `date` de formater la date donnée en argument à sa syntaxe par défaut, ou

```
user@PNS-VirtualBox:~$ date
lun. 17 avril 2023 16:48:36 CEST
user@PNS-VirtualBox:~$ date -d "2000-01-01"
sam. 01 janv. 2000 00:00:00 CET
user@PNS-VirtualBox:~$ date +"%Y/%m/%d" -d "2000-01-01"
2000/01/01
user@PNS-VirtualBox:~$
```

encore, en utilisant l'option « + », on peut modifier le format de la date que la commande doit nous retourner.

2.3 Aide sur les commandes

Il est impossible de connaître toutes les commandes par cœur, ni toutes les options et arguments possibles pour chacune des commandes. Heureusement, UNIX est bien fait et à tout prévu. Pour tout connaître sur une commande, une page de manuel est fournie avec le système (ces pages sont souvent disponibles dans plusieurs langues, mais la page en anglais est la référence). Pour tout savoir sur une commande :

```
$ man commande
```

Ce n'est pas toujours drôle de devoir lire le descriptif, mais c'est le passage obligé, même pour les plus chevronnés. Le site Web ci-dessous permet de savoir ce que fait une commande en fonction des options, ce qui peut être utile dans certains cas :

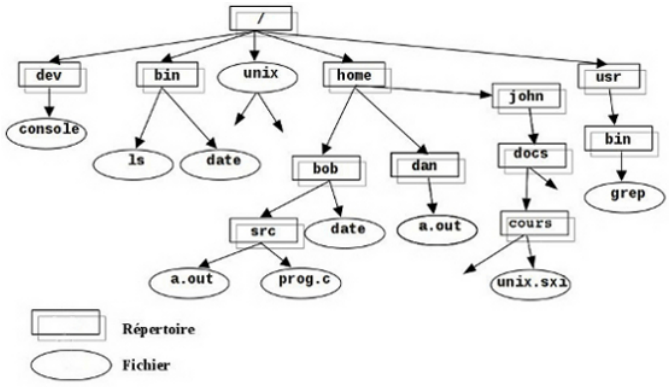
http://explainshell.com/

3 Répertoires

Un dossier ou répertoire est un objet informatique qui contient des fichiers. Et même les répertoires sont des fichiers, contenant une liste de fichiers et de sous-répertoires… un répertoire peut donc contenir :

- des fichiers;
- d'autres répertoires.

Le système de fichiers est organisé en une unique arborescence : les répertoires sont les nœuds de l'arborescence, la racine (*root directory*) est unique (contrairement à Windows) et notée « / », et les fichiers sont cités dans les répertoires.



Donc, en résumé :

Arborescence : Structure hiérarchisée des répertoires et des sous répertoires sur un disque dur, par exemple. Le tronc étant représenté par la racine du disque et les branches par les répertoires, etc. Il existe plusieurs chemins (sous UNIX) pour se rendre de la racine au répertoire voulu.

Racine : Point de départ d'une arborescence (notée « / » sous UNIX).

Homedir ($HOME) : C'est VOTRE répertoire personnel. Le répertoire personnel de l'utilisateur actuel est noté aussi « ~ », ou encore « ~user1 », supposant qu'on fasse référence au répertoire personnel de l'utilisateur « user1 ».

Répertoire : Une liste de descriptions de fichiers. Du point de vue du système de fichiers, il est traité comme un fichier dont le contenu est la liste des fichiers référencés. Un répertoire a donc les mêmes types de propriétés qu'un fichier comme le nom, la taille, la date, les droits d'accès et les divers autres attributs. Lorsque d'un répertoire, on veut aller au répertoire parent, celui-ci est désigné par « .. » sur la plupart des systèmes (on tapera donc « cd ..» sous UNIX pour accéder à un répertoire pa-rent).

Chemin (« path ») : Séquence de répertoires imbriqués, avec un fichier ou un répertoire à la fin, séparés par le caractère /

3.1 Chemins absolus et relatifs

Il y a 2 types des chemins : les chemins relatifs et les chemins absolus. Par exemple, si on se trouve dans le répertoire « dan » de l'arborescence montrée précédemment :

- le chemin relatif vers « a.out » est : « a.out »
- le chemin absolu vers « a.out » est : « /home/dan/a.out ». Un chemin absolu commence toujours depuis le répertoire racine du système (/)

3.2 Les différents types de répertoires basiques et spéciaux

Depuis le début d'Unix, aucune limitation majeure n'est faite quant à la longueur d'un nom de fichier. Tout caractère peut être utilisé dans le nom, et les extensions sont facultatives. Les différences de majuscules ou de minuscules constituent des fichiers distincts.

3.2.1 Le répertoire relatif courant : .

C'est le répertoire courant. Il est utilisé pour les commandes qui ont un répertoire comme argument. Il est également utilisé parfois pour lancer des commandes dans le répertoire courant.

3.2.2 Le répertoire relatif parent : ..

C'est le répertoire parent. Il fait toujours partie du répertoire. L'utilisation la plus courante est :

$ cd ..

3.2.3 Le répertoire absolu homedir : ~

Il n'est pas vraiment un répertoire spécial. Les interpréteurs de commande le remplacent juste par la valeur d'une variable de système appelé « HOME » de l'utilisateur courant.

3.2.4 Le répertoire absolu homedir : ~user1 (ou plus généralement ~login)

De façon analogue, il est remplacé par les shells par le répertoire utilisateur de l'utilisateur user1.

Si vous ne savez plus dans quel répertoire vous êtes, tapez la commande pwd et Linux vous l'indiquera !

3.3 Manipulation des répertoires

La commande `ls` (abréviation du mot list) affiche les fichiers et les sous-répertoires qui se trouvent dans un répertoire.

Les options de la commande ls

Option	Objectif
-a	(abréviation de mot « all » : tous) Affiche tous les fichiers en incluant ceux qui commencent par un point (i.e. les fichiers cachés)
-c	Affiche la dernière fois où le fichier a été modifié
--color	Affiche en couleur suivant le type des éléments
-l	(abréviation de mot « long ») Affiche en format long (type, date, taille, propriétaire, permissions)
-R	Affiche les contenus des répertoires du répertoire en cours
-S	(abréviation de mot « size » : taille) Liste les fichiers par taille (les fichiers les plus gros en premier)
-t	(abréviation de mot « temps ») Liste les fichiers selon la date de la dernière modification
-u	Liste les fichiers selon la date du dernier accès
-r	(abréviation de mot « reserve » : inversé) Affiche les fichiers en ordre inverse

Ces options peuvent être utilisées séparément ou combinées.

Par exemple :

-ltr	Format long, les fichiers les plus récents à la fin

Parmi les options de ce tableau, les options -a et -l sont les plus utilisées. Et la commande ls possède une cinquantaine d'options !

3.4 Comment se déplacer dans les répertoires ?

Pour se déplacer à l'intérieur de l'arborescence, on utilise la commande cd. Elle s'utilise en lui donnant le répertoire (absolu ou relatif) vers lequel on souhaite s'y rendre. E.g.

```
$ cd /etc/network
$ cd ../../tmp
```

Faites bien attention de séparer par un espace « cd » et le nom du dossier. UNIX exige une grande précision dans la syntaxe des commandes. Soumettez la commande au système grâce à la touche « Entrée », évidemment !

À la suite de cd on peut aussi utiliser - (tiret) qui désigne le répertoire précédent. Cela permet de revenir au répertoire précé-demment visité. Mais ce ne se fait que sur un seul niveau. Une fois revenu dans le répertoire précédent, - désigne celui d'où on vient.

```
$ cd -
```

4 Fichiers

Un fichier est une suite d'informations binaires, c'est-à-dire une suite de 0 et de 1. Ce fichier peut être stocké pour garder une trace de ces informations. Un fichier texte est un fichier composé de caractères stockés sous la forme d'octets.

Un fichier est enregistré sur le disque dur sous la forme « nom_du_fichier.ext ». « .ext » représente l'extension et c'est un moyen de reconnaître le type de programme avec lequel ce fichier peut être ouvert (attention cela ne garantit pas le type de fichier : lorsque l'on change l'extension on ne change pas le type de fichier !)

4.1 La commande « touch »

La commande « touch » fixe la date de dernière modification du fichier au moment présent. Si le fichier n'existe pas, la commande crée un fichier vide.

L'autre méthode dépend du type de fichier à créer. Si c'est un fichier texte, utilisez un des éditeurs de texte, par exemple `gnome-text-editor`.

4.1.1 Editer le contenu d'un fichier texte : gnome-text-editor

Vous pouvez créer ou éditer un fichier texte grâce à la commande gnome-text-editor. Notez que lorsque la commande gedit s'exécute (i.e. l'éditeur gedit est ouvert) vous ne pouvez plus exécuter de commandes dans le terminal ou vous avez exécutez gnome-text-editor. Pour exécuter de nouvelles commandes, vous devez donc soit :

- ouvrir un nouveau terminal (fenêtre ou onglet).
- Exécuter gnome-text-editor sous la forme : `$ gnome-text-editor &`

4.1.2 La commande cp ou copier des fichiers et des répertoires

La commande cp (abréviation du mot « copy ») copie un fichier existant vers un autre. Elle a deux arguments :

- le fichier à copier, qui doit exister et pouvoir être lu,
- le fichier résultat, qui doit être placé dans un répertoire où on a le droit d'écriture.
- Si le fichier résultat existe, le contenu original est écrasé par les nouvelles données copiées

Exemple :

```
$ cp  VariableAléatoire  Laplace
```

Si on met un nom de répertoire comme destination, le fichier copié a le même nom que le fichier à copier. Exemple :

```
$ cp  VariableAléatoire    khi2/
$ ls khi2/
  VariableAléatoire
```

ou :

```
$ cd   khi2/
$ cp   ../VariableAléatoire    .
$ ls
  VariableAléatoire
```

C'est la même chose au répertoire près !

La commande cp peut avoir plusieurs arguments :

```
$ cp   -r   VariableAléatoire   khi2/   LoiCauchy   LoiBernoulli   Probabilités/
```

(cp -r : copie récursive qui prend le répertoire et tout ce qu'il contient, y compris les sous-répertoires)

En règle générale, la commande cp a au moins deux arguments :

- n-1 premiers sont les noms de fichiers (ou de répertoires) à copier,
- le dernier est le nom du répertoire où copier,
- les noms sont systématiquement les mêmes.

Nous venons de voir que l'utilisation de cp est dangereuse et l'on risque parfois d'effacer (écraser) des fichiers importants. Les options de cp peuvent vous éviter des situations fâcheuses.

4.1.3 La commande mv ou déplacer, renommer des fichiers et des répertoires

La commande mv (abréviation du mot « move ») déplace un fichier ou dossier existant ou le renomme. Sous sa forme la plus simple, elle a 2 arguments :

- le fichier ou dossier source, qui doit exister et pouvoir être lu et supprimé,
- le fichier ou dossier destination, qui doit être dans un répertoire où on a le droit d'écriture.

Renommer un fichier, c'est le déplacer !

Si le fichier destination existe déjà, il est remplacé; si les 2 fichiers sont dans le même répertoire, cela revient à en changer le nom (renommer). Et comme pour la commande cp, il peut y avoir n arguments, le dernier étant un répertoire.

4.1.4 La commande mkdir

Pour créer un répertoire, utilisez la commande « mkdir ». « mkdir » reçoit comme argument le(s) nom(s) de(s) répertoire(s) à créer. Par exemple, pour créer un dossier « test » dans le dossier « /home/bob/ » existant, on peut exécuter « mkdir /home/bob/test ». Si jamais nous nous trouvions déjà dans le dossier « /home/bob/ », on aurait pu exécuter simplement « mkdir test ».

Si on exécute la commande « mkdir ~/test1/test11/ » pour créer le répertoire « test11 » alors que le répertoire « test1 » n'existe pas, la commande finira avec un message d'erreur et « test11 » ne sera pas créé. Lorsqu'on veut créer un répertoire, ainsi que son répertoire parent si ce dernier n'existe pas, on doit ajouter l'option « -p » à notre commande : « mkdir -p ~/test1/test11/ ».

4.1.5 La commande rm et rmdir

Pour supprimer un fichier, utilisez la commande rm (abréviation du mot « remove »), utilisée sans option et supprimant des fichiers spécifiés.

Les options de la commande rm

Option	Description
-f	Supprime des éléments sans demander de confirmation
-i	Demande confirmation avant de supprimer quoi que ce soit
-r	Accède aux sous-répertoires et supprime les éléments qui s'y trouvent
-v	Affiche des informations au cours du processus de suppression

Par exemple : « rm fichier1.txt » où « rm dir1/fichier1.txt »

Pour supprimer plusieurs fichiers, donnez uniquement la liste de tous ces fichiers à rm. E.g. « rm f1.txt f2.txt »

Pour supprimer un ou plusieurs répertoires vide, utilisez la commande rmdir. E.g.

```
$ rmdir dir1
```

où

```
$ rmdir dir1 dir2
```

Cependant, pour effacer un répertoire non vide, utilisez la commande rm avec l'option -r. Par exemple

```
$ rm -r dir1
```

5 Références à plusieurs fichiers- Globbing

Beaucoup de commandes acceptent une liste de fichiers dont les noms sont séparés par des espaces et constituent autant d'arguments. Le shell fournit une notation, dite de globbing, pour citer plusieurs fichiers à la fois. Les commandes

- le joker « * » représente une chaîne quelconque, éventuellement vide. Ex :
    - *.c
    - doc*unix.html
- le joker « ? » dénote un caractère quelconque. Les caractères « . » (en début de nom de fichier) et « / » ne peuvent pas être reconnus par un joker. Ex.,
    - ?nix
    - *.s??
- [...] représente un caractère appartenant à un ensemble
    - [Uu]nix
    - *.sx[ci]
- [^...] dénote un caractère n'appartenant pas à un ensemble
    - .[^co]
    - [^[:upper:]]*
- [] permet d'indiquer un intervalle des caractères également. Par exemple, l'intervalle de chiffres comprises entre 0 et 9 [0-9] ou [[:digit:]], l'intervalles de caractères alphabétiques en minuscule [a-z] ou [[:lower:]], majuscules [A-Z][[:upper:]], et autres variantes (e.g. [0-5],[a-dA-D], etc.)
- {} permet d'écrire 2 patterns de globbing distinctes
    - *.{c,o}
- !() trouve le complément d'une expression de globbing. Pour utiliser cette expression, il est nécessaire d'activer l'option « extglob » avec la commande « $ shopt -s extglob ».
    - !(*.o)

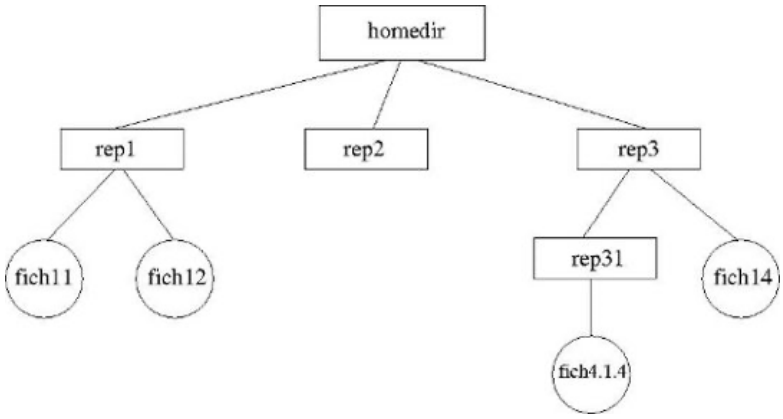
Le shell possède la caractéristique de terminer les noms de fichiers à votre place. Si vous écrivez un nom de fichier long sur la ligne de commandes, entrez les premières lettres, puis appuyez sur la touche Tab, le shell complète le reste ! C'est pareil pour les commandes.

---

6 Exercices

1. Testez la commande ls en affichant, depuis votre répertoire personnel initial (home directory), la liste de tous vos fichiers et sous-répertoires :
    a. sous un format condensé
    b. sous un format long (donnant le propriétaire, les permissions, la taille, ...)
    c. en affichant les fichiers cachés (dont le nom commence par un point)
    d. en ordre inverse
    e. avec un format long et en affichant les fichiers cachés, mais du plus récent au plus ancien
    f. avec un format long et en affichant les fichiers cachés, mais du plus ancien au plus récent
2. Où que vous soyez, quel est l'effet de la commande cd sans paramètre ? Faites plusieurs tests en vous exécutant « cd » depuis différents répertoires de votre système de fichiers.
3. Dans votre répertoire courant,
    a. créez en une commande les fichiers suivants : annee1 annee2 Annee3 f1.txt f2.txt f11.txt
    b. Créez le répertoire Year dans votre répertoire courant. Puis, en une commande déplacez les fichiers « annee1 annee2 An-nee3 », sans utiliser la syntaxe globbing, dans le répertoire Year. Ensuite, en utilisant la syntaxe globbing, déplacez « f1.txt, f2.txt » dans Year.
4. Créez un répertoire system sous votre dossier personnel, puis un répertoire tp1 sous system
5. Essayez d'effacer le répertoire system avec la commande rmdir. Que constatez-vous ?
6. Effacez avec la commande rm le dossier « system » et créez à l'aide d'une seule commande les répertoires system, system/tp1, system/tp2
7. Renommez le répertoire system en test
8. Copiez un fichier de votre choix du répertoire /bin dans le répertoire test/tp1 de votre dossier personnel :
    a. En faisant la copie depuis le répertoire /bin
    b. En faisant la copie depuis le répertoire test/tp1
    c. En faisant la copie depuis votre homedir, en utilisant des chemins absolus
    d. En faisant la copie depuis votre homedir, en utilisant des chemins relatifs
9. Effacez à l'aide d'une seule commande les répertoires test/tp1 et test/tp2

Soit l'arborescence suivante :



10. Comment déplacer toute l'arborescence rep3 sous le répertoire rep2 ?
11. Comment copier tous les fichier fX.txt (où X est un numéro entier entre 0 et 5) du répertoire courant vers le dossier /tmp
12. Créez les fichiers f1A.txt, f1a.txt, f1a.aux, f11.txt, f1a.TXT, f2A.txt f2a.txt, f21.txt, f2a.TXT, f2a.aux dans « rep2 ». Comment lister avec la méthode du globbing
    a. Tous les fichiers finissant par l'extension .txt et .TXT ?
    b. Tous les fichiers dans commençant par f1, dont le 3èmer caractère n'est pas un chiffre ?
    c. Tous les fichiers dans le troisième caractère est un caractère alphabetique minuscule ?
    d. Listez tous les fichiers ne finissant pas avec l'extension .aux

7 Synthèse

7.1 Exercice A :

Déterminez les commandes permettant de réaliser les actions suivantes :

- Déterminer le répertoire par défaut dans la hiérarchie des répertoires ?
- Y a t-il des fichiers, des répertoires dans ce répertoire ?

ls -l (repérer les noms dont le premier caractère est un d pour directory)

- Entrer du texte dans un fichier nommé « Mon_fichier » que vous avez créé au préalable.
- Lister le contenu de « Mon_fichier ».
- Lister le répertoire courant.
- Lister les répertoires /bin et /dev.
- Créer sous votre répertoire deux sous-répertoires : « Source » et « Data ».
- Se positionner sous « Source ».
- Listez le répertoire courant.
- Revenir sous le répertoire de départ et détruire « Source ».
- Créer un deuxième fichier nommé « Mon_fichier_2 ».

- Copier chaque fichier en nom_de_fichier.old.
- Créer un répertoire « Old ».
- Déplacer les fichiers avec l'extension .old vers le répertoire « Old ».
- Effacer tous les fichiers crées dans Old sans effacer le répertoire Old.

7.2 Exercice B

Exploration de l'arborescence Linux.
ls, cp, mv, rm, cd, pwd, mkdir, rmdir

- Indiquez par une commande dans quel répertoire vous vous trouvez.
- Allez dans le répertoire /usr/share/doc, puis vérifiez le chemin de votre répertoire courant.
- Remonter dans le répertoire parent puis vérifier.
- Allez dans votre répertoire personnel sans taper son chemin.
- Retournez dans votre répertoire précédent sans taper son chemin.
- Retourner dans votre répertoire personnel et listez les fichiers présents.
- Listez maintenant tout les fichiers (même ceux cachés).
- Affichez de façon détaillée le contenu du répertoire /usr sans changer le répertoire de travail.
- Affichez l'arborescence de fichiers contenue dans /var sans changer le répertoire de travail.
- Affichez de façon détaillée le contenu du répertoire /var/log en classant les fichiers du plus vieux au plus récent.

7.3 Exercice C

Répertoires et consultation de fichiers.

- Allez dans votre répertoire personnel.
- Créez un répertoire portant le nom de CommandesLinux.
- Allez dans votre répertoire CommandesLinux.
- Créez l'arborescence cours1/cours2/cours3/cours4.
- Listez le contenu du répertoire courant de façon récursive.
- Supprimez le répertoire cours1. Est-ce possible ?
- Supprimez l'arborescence de répertoire cours1/cours2/cours3/cours4, puis vérifiez en listant le répertoire de façon récursive.
- Allez dans le dossier CommandesLinux et créez les répertoires suivants :

Code :

```
.
 |--couleur
 |  `--froide
 `--forme
   |--angle
   `--courbe
```

- Copiez le fichier /etc/services dans votre répertoire CommandesLinux.
- À qui appartient le fichier que vous venez de copier ? Quelle est sa date de sa dernière modification ?
- Créez les fichiers ne contenant aucune donnée et dont les noms sont les suivants : rond.txt, triangle.txt, carre.txt, rectangle.txt, vert.txt et bleu.txt
- Déplacez le fichier rond.txt dans le répertoire courbe et les fichiers triangle.txt, carre.txt, rectangle.txt dans le réper-toire angle.

- Déplacez les fichiers vert.txt et bleu.txt dans le répertoire froide.
- Allez dans le répertoire couleur et afficher le contenu du répertoire de façon récursive.
- Copier le répertoire sous le nom chaude. Est-ce possible ? Comment ?
- Allez dans le répertoire chaude et renommez le fichier bleu.txt en rouge.txt et vert.txt en jaune.txt.
- Remontez dans le répertoire CommandesLinux et renommez le répertoire couleur en peinture. Est-il besoin de spéci-fier une option particulière à la commande mv.
- Listez la totalité de l'arborescence contenue dans le répertoire CommandeLinux.
- Affichez le contenu du fichier /etc/issue. Que contient-il ?